

Лабораторная работа № 5

Тема:

1. Веб-программирование;
2. Веб-фреймворк Django

Цель работы:

Закрепить знания и навыки по работе со стандартными библиотеками, фреймворком Django.

Задание.

Реализовать веб-сайт с использованием Веб-фреймворк Django по индивидуальному заданию (выбрать вариант по номеру в журнале или реализовать свой проект - согласовать с преподавателем до начала разработки)

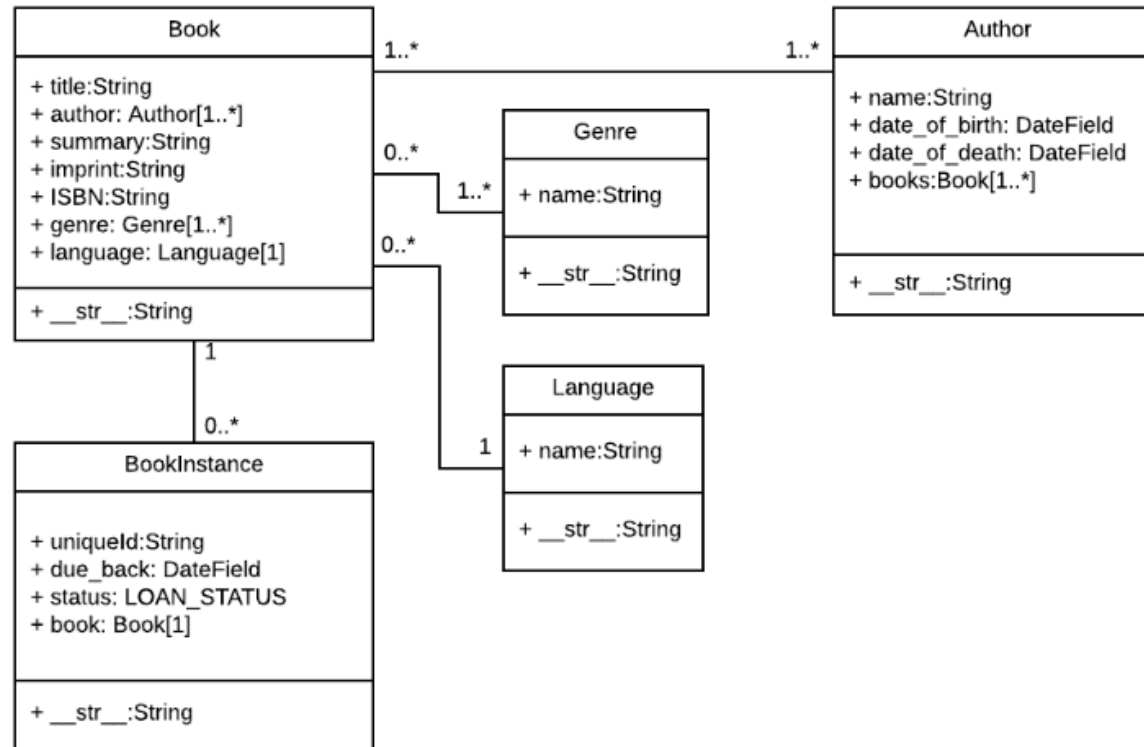
Постановка задачи:

- Определить необходимые сущности для описания предметной области. Реализовать их в терминах моделей, используя подходящие типы данных и связи объектов <https://developer.mozilla.org/ru/docs/Learn/Server-side/Django/Models>. Представить на диаграмме в виде, как на примере ниже. **В качестве базы данных использовать sqlite.** Нужно предусмотреть возможность хранения и изменения данных для обеспечения функционала в соответствии с индивидуальным заданием и для страниц, которые должны быть в каждом приложении независимо от варианта. Эти страницы полностью будут реализованы в следующем

семестре, поэтому сейчас можно ограничиться их наполнением через базу данных или админ панель, или выполнить по желанию(отмечено ***):

- *Главная*: Краткая информация о последней опубликованной статье
- *О компании*: информация о компании – просто текст (** видео, логотип, история по годам, реквизиты – пока только сделать таблицу в базе данных) ([таблица в базе данных](#))
- *Новости*: список статей в соответствии с тематикой сайта с заголовком, кратким содержанием (одно предложение), картинкой ([таблица в базе данных](#)) (** и кнопкой «Читать далее» при нажатии на которую открывается вся статья)
- *Словарь терминов и понятий*: список часто задаваемых вопросов и ответов на них с датой добавления на сайт ([таблица в базе данных](#))
- *Контакты*: Фото сотрудников с описанием выполняемых работ, телефонами, почтой и т.д. ([таблица в базе данных](#))
- *Политика конфиденциальности*: пока добавить пустую страницу
- *Вакансии*: список вакансий с описанием ([таблица в базе данных](#))
- *Отзывы*: список отзывов с указанием имени, оценки, текста, даты ([таблица в базе данных](#)). (Кнопка добавить отзыв с переходом к окну регистрации или в личный кабинет залогиненного пользователя. При нажатии кнопки открытие формы с полем для текста отзыва и выбора оценки, кнопкой «Отправить», которая сохраняет отзыв в базе.)
- *Промокоды и купоны*: список промокодов действующих и в архиве. ([таблица в базе данных](#)).

- Реализовать связи один к одному (OneToOneField), один ко многим (ForeignKey) и многие ко многим (ManyToManyField).



- Реализовать CRUD (create, read, update, delete) операции (<https://metanit.com/python/django/5.4.php>);
- Добавить все модели в админ панель, создать суперюзера, обеспечить работу с данными, фильтрацию, встроенное редактирование связанных записей (https://developer.mozilla.org/ru/docs/Learn/Server-side/Django/Admin_site).
- Реализовать механизмы авторизации/аутентификации;

- Разграничить доступ в зависимости от прав: например, владелец магазина(superuser), User с регистрацией, User без регистрации (см. инд. задание)
- Обеспечить наполнение данными для демонстрации не менее 10 записей в списке товаров/услуг/объектов/клиентов
- Подключить и использовать как минимум 2 сторонние API (базовый список прилагается), какие именно - выбрать на свой вкус.
- Использовать регулярные выражения для связи URL-адреса с функцией отображения (https://developer.mozilla.org/ru/docs/Learn/Server-side/Django/Generic_views)
- отображать статистические показатели для своего сайта (использовать данные, соответствующие предметной области). Например:
 - список клиентов/товаров в алфавитном порядке и общую сумму продаж;
 - статистические показатели (среднее, мода и медиана) по сумме продаж;
 - статистические показатели (среднее и медиана) по возрасту клиентов;
 - какой тип товаров наиболее популярен?
 - какой тип товаров приносит наибольшую прибыль?
- отображать тайм зону пользователя, текущую дату, дату добавления/изменения данных в таблицы для тайм зоны пользователя и для UTC, календарь в текстовом виде. Дату указывать в формате (DD/MM/YYYY)
- Номер телефона клиента указывается в формате +375 (29) XXX-XX-XX;
- Клиенты и сотрудники должны иметь возрастное ограничение 18+;
- Реализовать визуализацию в виде диаграммы или графика для отображения распределения показателей по группам и/или изменения показателей по датам/группам.
- Реализовать поиск по любому из параметров и сортировку отображаемых данных

- Добавить тесты. Покрытие тестами кода на 80% и выше (<https://developer.mozilla.org/ru/docs/Learn/Server-side/Django/Testing>) (<https://pytest-docs.ru.readthedocs.io/ru/latest/parametrize.html>);
- Добавить logging(<https://habr.com/ru/companies/wunderfund/articles/683880/>);
- Внешний вид сайта не важен, главное отображать нужную информацию. Использование css по желанию. Если будете использовать <https://www.bootstrapcdn.com/>, то сохраните на github в разных ветках версию без стилей и со стилями. В следующем семестре разработаете свои стили и нужен будет проект без bootstrap;
- Валидация форм как на стороне сервера, так и на стороне клиента;
- Поддержка разного уровня логирования (уровень логирования брать из конфигурации приложения);
- Ограничить использование API проекта для неавторизованных запросов.
- **Выполнить хостинг сайта, используя один из следующих ресурсов (бесплатные):**
 1. [Render.com](https://render.com) (Лучший баланс простоты и функциональности)
 2. [PythonAnywhere.com](https://pythonanywhere.com) (Классический вариант, проще, но с ограничениями)
 3. **Railway.app** (Очень гибкий и мощный, чуть сложнее)
 4. [Fly.io](https://fly.io) (Хорош для производительности, немного сложнее в настройке)
 5. **Heroku** (Имеет бесплатный тариф, но требует привязки карты)

Ваш проект должен использовать **PostgreSQL** в продакшене (лучшая практика). Для локальной разработки использовать SQLite.

- Разработать Dockerfile для вашего проекта (или частей проекта);
- docker-compose файл, с помощью которого возможно запустить проект локально (должен включать образы вашего проекта (образы сделать публичными, чтобы преподаватель мог запустить у себя) и базы данных + возможно какие-то сервисы, необходимые для проекта);
- **обязательно проект должен быть на гитхаб**. Адрес для организации моего доступа к закрытым репозиториям с выполненными ЛР на GitHub @AnnBsuir (anzh52889@gmail.com)

- ***** дополнительное задание** – по желанию тем, кто стремится освоить больше и получить +балл по ЛР:
 - Дополнить проект модулем для работы с параллельным кодом (можно использовать asyncio, multiprocessing или multithreading на выбор); Добавить отдельную страницу сайта без css оформления для демонстрации решения задачи. Вариант индивидуального задания тот же, что и в начале семестра - см. в самом конце данного файла

Примечание: если проект будет не только в вашем гит репозитории, то результат лабораторных в этом семестре обнуляется.

Доработка html, css, js будет выполнена на первых трех ЛР в курсе «Современные технологии разработки web-приложений», поэтому обеспечьте сохранность проекта до следующего семестра.

Для справки см. также инфо в СЭО:

Список бесплатных API: <https://lms.bsuir.by/mod/resource/view.php?id=214284>

Как создать проект в Django и использовать стороннее API

<https://lms.bsuir.by/mod/resource/view.php?id=214285>

Варианты заданий:

№	Web сайт	Ориентировочные модели	Описание (что должно быть обеспечено)	Отображается для Админа (Superuser)	Отображается для User с регистрацией – личный кабинет + все что видит User без регистрации	Отображается для User без регистрации
			имеет некоторое название, цену характеристики. Для проведения исследований и оптимизации работы магазина Вы пытаетесь собирать данные с Ваших клиентов. При этом для Вас определяющее значение имеют стандартные анкетные данные, а также телефон и адрес электронной почты для связи. По каждому факту продажи Вы автоматически фиксируете клиента, товары, количество, дату продажи, дату доставки.	Ежемесячный объем продаж товаров каждого вида; Годовой отчет поступлений от продаж; Анализ данных должен включать следующие элементы: Сводные таблицы и диаграммы по клиентам, видам продукции, объемам и поступлениям от продаж; Прогноз продаж; Построение линейного тренда продаж	User с регистрацией – клиент – имеет возможность купить услуги, видит инфо о покупках и промокодах	
13.	Риэлтерское агентство	Объекты недвижимости, Категории, Владельцы, Покупатели, Сотрудники, Продажи	Вы являетесь сотрудником коммерческого отдела риэлтерского агенства, продающей объекты недвижимости через Интернет. Вашей задачей является отслеживание финансовой составляющей работы компании. Работа Вашей компании организована следующим образом: на Интернет-сайте компании представлены (выставлены на продажу) объекты недвижимости. Каждый из них имеет некоторое название, цену и характеристики Для проведения исследований и	добавлять и редактировать информацию о объектах недвижимости, клиентах, сотрудниках компании, сделках. вывод определенного объекта недвижимости, статистики по сотрудникам, стоимости и суммы сделок с операциями по недвижимости и т.д.	User с регистрацией – сотрудник – видит инфо о сделках, объектах недвижимости и клиентах с которыми работает User с регистрацией –	инфо о категориях оказываемых услуг и самих услугах с фильтрацией по цене/категории и т.д. , промокоды, купоны

№	Web сайт	Ориентировочные модели	Описание (что должно быть обеспечено)	Отображается для Админа (Superuser)	Отображается для User с регистрацией – личный кабинет + все что видит User без регистрации	Отображается для User без регистрации
			оптимизации работы агентства Вы пытаетесь собирать данные с Ваших клиентов. При этом для Вас определяющее значение имеют стандартные анкетные данные, а также телефон и адрес электронной почты для связи. По каждому факту продажи Вы автоматически фиксируете клиента, дату продажи, дату договора.		клиент – имеет возможность купить недвижимость, видит инфо об объектах, покупках и промокодах	
14.	Аптека	Медикаменты, Сотрудники, Продажи, Поставщики, Отделы аптеки, Категории медикаментов	Автоматизация работы аптеки, основным направлением которой является продажа медикаментов разных видов	добавлять и редактировать информацию о медикаментах(код, название, инструкция, описание, стоимость, фото), отделах аптеки, сотрудниках и поставщиках. информацию о всех продажах выбранных медикаментов. информация о конкретном лечащем средстве, о конкретном сотруднике, общей выручке со всех продаж и по отделам.	User с регистрацией – сотрудник – видит инфо о продажах и поставщиках с которыми работает User с регистрацией – покупатель – имеет возможность купить товар, видит инфо о точках самовывоза, покупках и промокодах	Инфо о категориях медикаментов и самих медикаментах с фильтрацией по цене , доп. услугах, промокоды, купоны
15.	Медицинский центр	Клиенты, Диагнозы, Врачи,	Клиентом медицинского центра является любой человек, который хотя бы раз	актуальный список оказываемых услуг;	User с регистрацией –	инфо о категориях оказываемых

31. Детальная настройка полей формы
32. Стилизация полей форм
33. Модели в Django
34. Подключение к базе данных
35. Создание моделей
36. Типы полей моделей
37. QuerySet API
38. Создание и получение объектов модели
39. Редактирование и удаление объектов модели
40. Фильтрация
41. values и values_list и сортировка
42. Операции с множествами
43. Получение отдельных объектов и проверка их наличия
44. Агрегатные операции
45. Выполнение SQL-выражений
46. CRUD. Все базовые операции с моделями в веб-приложении
47. Отношение один ко многим (One to Many)
48. Практический пример связи один ко многим
49. Отношение многие ко многим (Many to Many)
50. Отношение многие ко многим (Many to Many)
51. Отношение один к одному (One to one)

Дополнительное задание лабораторной работы 5:
МНОГОЗАДАЧНОСТЬ, КОНКУРЕНТНОСТЬ И АСИНХРОННОСТЬ В PYTHON

1. СПИСОК ИСТОЧНИКОВ ДЛЯ ДОПОЛНИТЕЛЬНОГО ИЗУЧЕНИЯ

Для глубокого погружения в тему рекомендуется изучить следующие материалы:

- **Базовые концепции:** [Senjun.ru](https://senjun.ru) — Многопоточность и процессы в Python
- **Сравнение подходов (Sync vs Async):** [DZone — Python Async vs Sync Blocking](https://dzone.com/articles/python-async-vs-sync-blocking)

- Детекция блокировок в Asyncio: [DZone — Advanced Blocking Detection](#)
- Синхронизация в асинхронной среде: [DZone — Synchronization with Asyncio](#)
- Официальный концептуальный обзор: [Python Docs — Conceptual Overview of Asyncio](#)
- Русскоязычный гид: [Python Academy — Введение в Asyncio](#)

Ниже представлено углубленное описание теоретического минимума с подробными пояснениями, примерами кода и анализом механизмов работы каждого инструмента.

2. ПОДРОБНЫЙ ТЕОРЕТИЧЕСКИЙ МИНИМУМ

2.1. Конкурентность vs Параллелизм

Понимание разницы между этими понятиями — ключ к выбору правильной архитектуры приложения.

Конкурентность (Concurrency) — это искусство управления множеством задач одновременно. Представьте повара на кухне: он нарезает овощи, но каждые 2 минуты отвлекается, чтобы помешать суп, а затем проверяет духовку. Он не делает всё это в одну и ту же наносекунду, но за счет быстрого переключения создается прогресс по всем направлениям. В программировании это "иллюзия одновременности", реализуемая через переключение контекста процессором.

Параллелизм (Parallelism) — это физическое выполнение нескольких задач в один и тот же момент времени. Это возможно только в многоядерных системах. Если на кухне работают два повара, один может *одновременно* с другим резать лук. Здесь нет переключений — задачи выполняются физически независимо.

2.2. Типы задач: I/O-bound и CPU-bound

Эффективность многозадачности напрямую зависит от того, какой ресурс является "бутылочным горлышком".

1. **I/O-bound (Input/Output bound):** Задачи, скорость которых ограничена внешними системами. Сюда относятся:
 - Запросы к API или базам данных.
 - Чтение и запись больших файлов на диск.
 - Ожидание ввода пользователя.